

Reconfigurable Digital Integrated Circuit Test Fixture User Manual



I. INTRODUCTION

This user manual provides clear instructions for setting up and using the **Reconfigurable Digital Integrated Circuit Test Fixture**. This project requires the user to fabricate and populate the PCB and build the associated software. The user should have basic experience with electronics and software development tools. This guide covers the required dependencies, building and compiling the project, flashing the STM32, and executing the application to test a digital integrated circuit along with an interpretation of the PDF report generated.

II. PREREQUISITES

This project is distributed as source code and schematics, the hardware must be procured and constructed by the user themselves, and the software must be compiled and flashed.

A. Hardware

Printed Circuit Board Fabrication

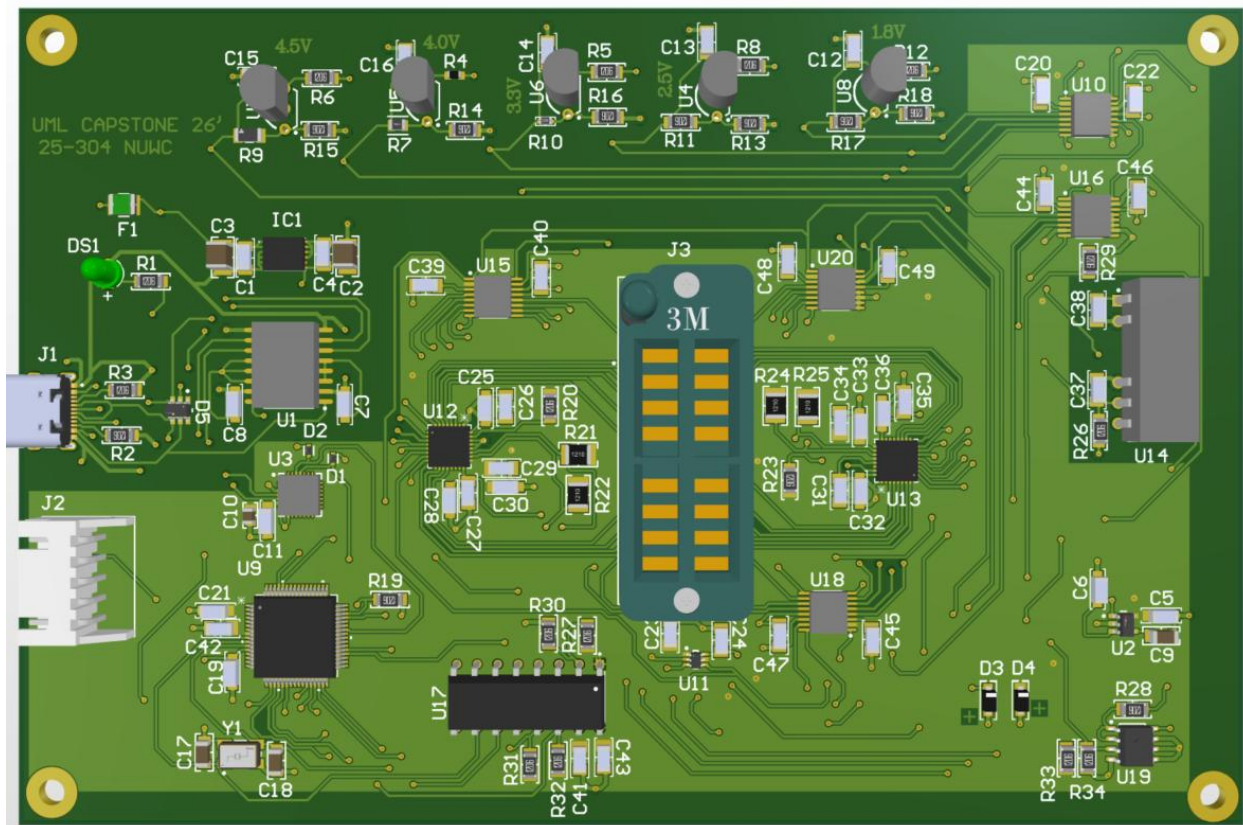


Figure 1. 3D PCB view of the hardware portion of the test fixture.

The Bill of Materials (BOM) is located in the docs/ folder. The file is an Excel Spreadsheet and can be opened using Microsoft Excel, or Google Sheets,

Gerber and ODB++ files for PCB fabrication can be found in hardware/altium_files. It is recommended to order the PCB with a stencil for easier population. The Engineering Change Order (ECO) is located in docs/. The ECO indicates all changes that have been made to the PCB since the initial fabrication and should be referenced during population.

Chassis

The chassis for the test fixture are located in hardware/chassis. This folder includes all the SolidWorks design files used to fabricate the chassis from the original PolyCase.

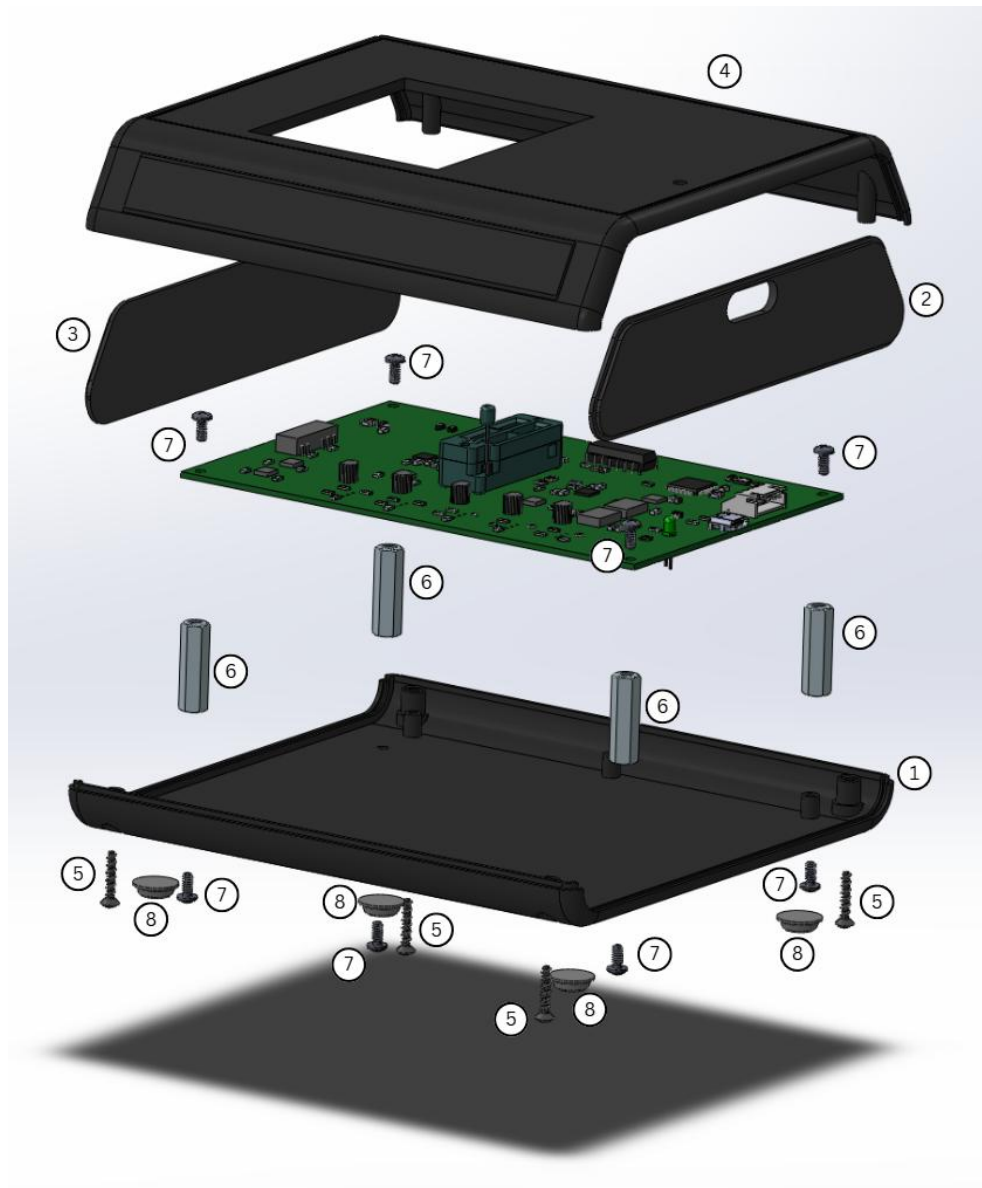


Figure 2. 3D Chassis scatter view of the IC Test Fixture.

Figure 10 showcases the assembly component view of all parts required to assemble the chassis of the reconfigurable test fixture and set the PCB properly inside. The chassis comes with on bottom chassis (1), a front panel with cutout (2), rear panel (3), and upper chassis with cutouts (4). These can be taken apart and secured together with a snap fit assembly for the panels and secured in place with the included chassis screws x4 (5). Four adhesive rubber feet (8) are attached onto indent holes on the bottom of the chassis. These provide a non-slip grip to a flat surface and prevent the chassis from moving around and damaging surfaces. Four 1” hex standoffs (6) are used to hold up the PCB and can be secured into place using four #4-40 screws (7). It is best practice to secure the hex standoffs in place first by screwing them in place from the bottom of the chassis. Then mount the PCB on top with the USB-C port facing towards the front panel side, then secure the PCB in place with the remaining four #4-40 screws (7). Finally, the front (2) and rear (3) panels can be placed into the indentation marks of the bottom chassis, and the upper chassis (4) can be placed on to, with the LED hole aligned with the PCB LED. The chassis can be secured together using the four included chassis screws (5).

Drivers

The circuit uses a CP2102 USB to UART bridge for communication. The firmware for the CP2102 requires a driver provided by Silicon Labs which can be downloaded from <https://www.silabs.com/interface/usb-bridges/classic/device.cp2102?tab=softwareandtools>. The application finds the test fixture dynamically by the CP2102’s serial number which is flashed using another software provided by Silicon Labs which can be downloaded from same link and is called “AN721: CP210x/CP211x Device Customization Guide.” The serial number must be programmed to 554D4C2043617073746F6E652032352D333034204E5557432032303236.

A USB C 2.0 cable is also required for communication with the test fixture.

B. Software

The software application is programmed in Python. The user must have their own Python interpreter that is at least version 3.10 or greater. The current software was built and tested using Python 3.14.3. A Python interpreter can be installed from <https://www.python.org/downloads/>. To create a Python virtual environment, follow the terminal commands displayed in Figure 2.

```
cd software/  
  
python -m venv .venv  
  
# linux/mac  
source .venv/bin/activate  
  
# windows  
.venv\scripts\Activate.ps1
```

Figure 3. Terminal commands for creating Python virtual environment.

To install the project dependencies, either command in Figure 3 can be used. It is recommended to install the developer dependencies if it is desired to modify the source code.

```
# run in same directory as pyproject.toml  
  
# only required dependencies  
pip install -e .  
  
# install with additional optional dev dependencies  
pip install -e .[dev]
```

Figure 4. Terminal commands for installing Python package dependencies.

This program can be compiled using the provided scripts in software/build to build an executable for Windows, Mac, or Linux, or installed as a package. To build an executable, commands shown in Figure 4 can be used.

```
# linux/mac
bash ./scripts/build.sh

# windows (PowerShell)
powershell -ExecutionPolicy Bypass -File .\scripts\build.ps1
```

Figure 5. Terminal commands for building an executable of the application.

If an executable is not desired, or the user is working in a development environment, the software can be executed using the commands in Figure 5.

```
# run directly from src
python src/ic_test_fixture/main.py

# or if installed in editable mode
python -m ic_test_fixture.main
```

Figure 6. Terminal commands for executing the application.

C. Firmware

The firmware is programmed in C and must be flashed to the STM32 on the PCB using an ST-Link debugger/programmer. All related firmware is located in `firmware/`. The PCB has a 5-pin header (J2), located at the bottom right as seen in Figure 1. See Figure 7 for proper connections between the ST-Link and header, jumper wires or a proper link may be used.

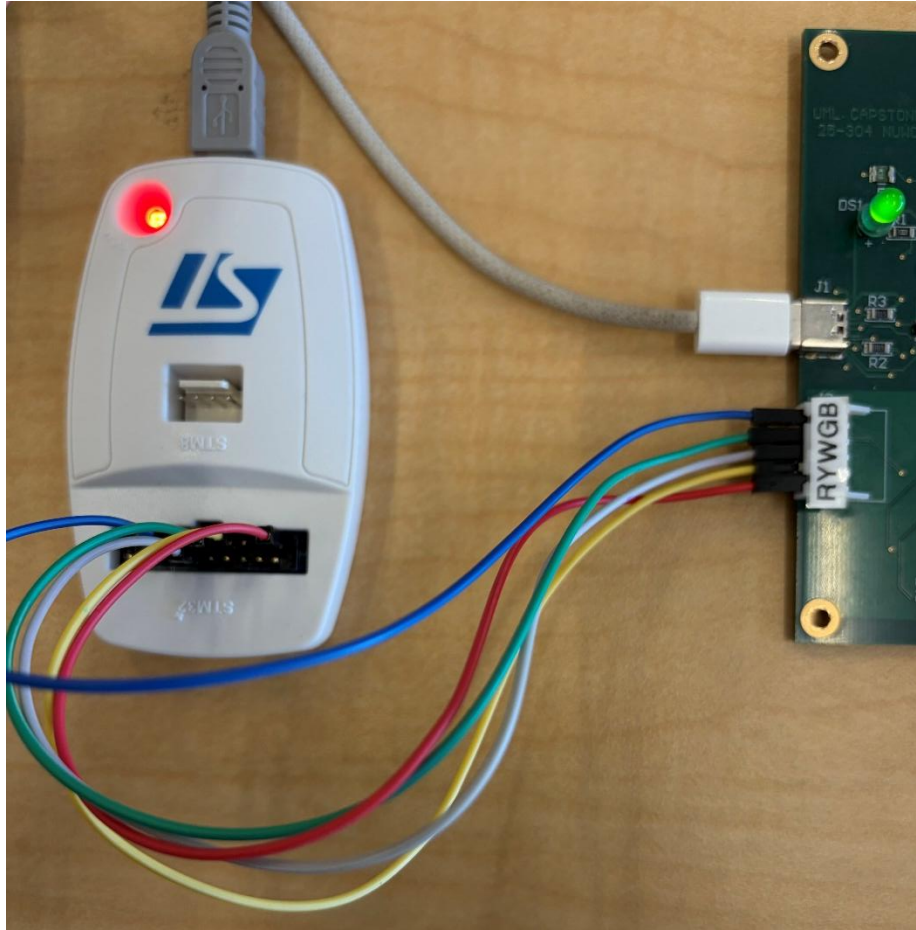


Figure 7. Jumper wire connections between the ST-Link and 5-pin header for flashing.

To build and flash the firmware, the following directions must be followed:

1. Open STM32CubeIDE
2. File -> Open Projects from File System
3. Import the project from firmware/STM32_main
4. Project -> Build Project
5. Connect ST-Link to PCB (shown in Figure 6)
6. Power the board via USB-C 2.0 cable
7. Run → Debug (or Run)

III. IC Test Fixture Application

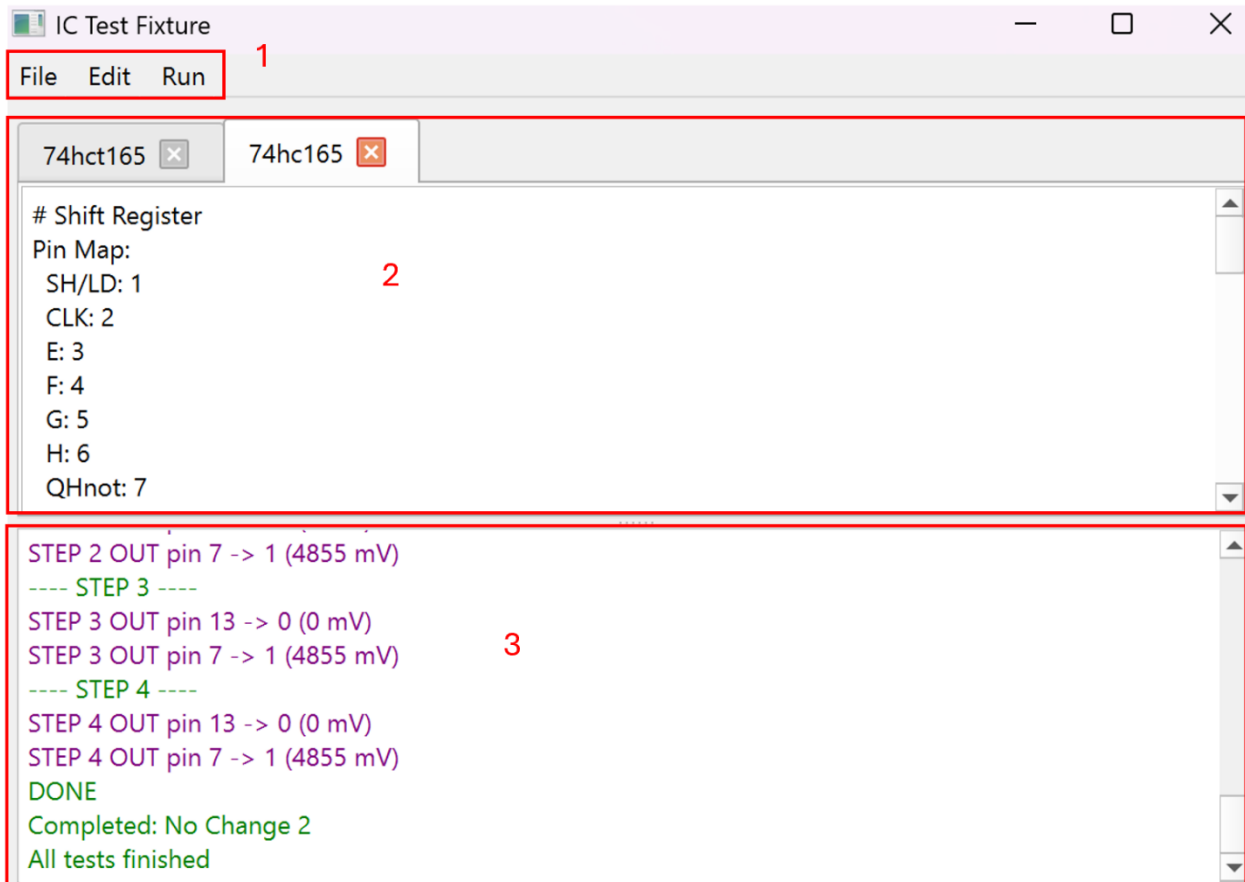


Figure 8. Labeled image of the main window of the application.

The IC Test Fixture application is a graphical-user interface, allowing the user to control the test fixture. The main window of the application has three primary features shown in Figure 8, the menu bar (1), the text editor (2), and the status display (3).

The menu bar includes the menus File, Edit, and Run. The File menu offers the user the ability to make a new test script, open an existing test script, and save a test script. The Edit menu contains standard text-editing features undo, redo, cut, copy, and paste. The Run menu contains a button Run, and BIST. The Run button is used for executing a test script, and BIST is used to execute the Built-In Self Test feature of the hardware.

The text editor has support for tabs. The tabs allow the user to swap between multiple opened test scripts to streamline the process of testing multiple digital integrated circuits (ICs). All

editing and test script executing features are tied to the current tab in focus. The text editor only supports plaintext.

The status display displays all commands sent and received by the hardware. The status display is color-coded to easily identify the types of status messages. See Table 1 for what each color stands for. The status display acts as a pseudo-log for debugging purposes if the application is used in a development environment.

Table 1. The table displays the meaning of each color of text in the status display.

Color	Command Type
Blue	Commands sent to the test fixture.
Red	Errors with the test script or received from the test fixture.
Green	Miscellaneous status messages regarding current progress.
Purple	Measurements received from the test fixture, indicating the measured voltage and logic based on the given thresholds.

IV. YAML Test Scripts

The primary method for a user to control the test fixture is using YAML test scripts. For an in-depth guide on YAML test scripts, refer to the document “YAML Test Script Documentation” located in the docs/ folder. The test scripts allow the user to set the supply voltage for the IC (VCC), and dictate which pins to drive with a digital logic signal, and which pins to measure. The test scripts support both high-level abstraction for readability, and low-level logic for simplicity. It is recommended to use high-level abstraction for ICs with complex logic. There are five sections to the test script, Chip Info, Pin Map, Truth Table, Global Parameters, and Tests. The first three sections are optional; Global Parameters and Tests are required.

Chip Info

```
Chip Info:
Name: 74HCT00
Manufacture: Texas Instruments
Logic: NAND
Datasheet: https://www.ti.com/lit/ds/symlink/sn74hct00.pdf
```

Figure 9. Example Chip Info section for 74HCT00 IC.

The Chip Info section allows the user to input any additional information regarding the IC if desired. There are no restrictions to this section and none of the information written will interfere with the testing of the chip. All the information will be displayed in the PDF report in a tabular format. This section is ideal if the PDF report is intended to be passed onto someone else who may want additional information. An example is shown in Figure 9.

Pin Map

```
Pin Map:
A1: 1
A2: 4
A3: 9
A4: 12
B1: 2
B2: 5
B3: 10
B4: 13
Y1: 3
Y2: 6
Y3: 8
Y4: 11
```

Figure 10. Example Pin Map Section for 74HCT00 IC.

The Pin Map section allows the user to abstract the pin numbers of the socket to the actual pin names of the IC. These abstractions can then be used in the Tests section of the test script to enhance readability for both the test script and generated PDF report. This section is recommended for ICs with high pin counts. An example is shown in Figure 10.

Truth Table

```
Truth Table:
- {A: L, B: L, Y: H}
- {A: L, B: H, Y: H}
- {A: H, B: L, Y: H}
- {A: H, B: H, Y: L}
```

Figure 11. Example Truth Table section for 74HCT00 IC

The Truth Table section allows the user to write a truth table like data structure inside the test script. The name of a column of the Truth Table can later be used in the Tests sections to abstract the logic signal to drive a pin. This section is useful for logic devices that have multiple independent logic circuits. Each logic circuit can then be tested and presented separately in the generated PDF report. An example is shown in Figure 11.

Global Parameters

```
Global Parameters:
VCC Pin: 20
GND Pin: 7
VCC Voltage: 5
Output Low: 0.33
Output High: 3.84
```

Figure 12. Example Global Parameters section for 74HCT00 IC.

The Global Parameters section lists all the parameters that will be used for every test. VCC Pin and GND Pin represent the voltage supply pin and ground pin of the IC respectively. VCC Voltage is the voltage level to power the IC, and the default logic HIGH voltage during testing. The parameters Output Low and Output High are the logic thresholds used to determine if the output voltage is HIGH, LOW, or undefined. An example is shown in Figure 12.

Global Parameters:

VCC Pin: 20

GND Pin: 7

VCC Voltage: [2.5, 3.3, 4.0, 4.5, 5]

Output Low: [0.3, 0.3, 0.3, 0.3, 0.3]

Output High: [2.2, 3.0, 3.6, 4.1, 4.7]

Figure 13. Example Global Parameters section for 74HC00 IC.

To streamline the process of testing logic devices capable of operating at different voltage levels such as the 7400-series HC devices, square brackets can be used to create list of voltage levels and thresholds. These lists must be the same length. The voltage levels and thresholds located at the same index correspond to each other during testing, that is, the first VCC Voltage will have thresholds at the first Output Low and Output High values. An example is shown in Figure 13.

The voltage levels the IC test fixture can support are 1.8V, 2.5V, 3.3V, 4.0V, 4.5V, and 5.0V.

Tests

Tests:

A=L, B=L -> Y=H:

Inputs:

1,4,15,18: L

2,5,16,19: L

Outputs:

3,6,14,17: H

Figure 14. Example of a test for 74HCT00 IC.

The Tests section is where the user will be able to write commands to drive the input pins of the IC and measure the output pins. Pins can be logically grouped together by separating pins with a comma. In Figure 14, the “A” input pins 1, 4, 15, and 18, and “B” input pins 2, 5, 16, and 19, are driven with a logic LOW. The “Y” output pins 3, 6, 14, and 17 are expected to be a logic HIGH.

```
Tests:
  NAND 1:
    Inputs:
      A1: A
      B1: B
    Outputs:
      Y1 : Y
```

Figure 15. Example of test for 74HCT00 IC using abstractions.

If the Pin Map or Truth Table section were included in the test script, the pin numbers and logic symbols can be abstracted for actual names of the pins for the IC, and the column name of a truth table. In Figure 15, the pin A1, B1, and Y1 represent the pins 1, 2, and 3 as shown in Figure 10. The values to drive and measure the pins with are based on the NAND gate truth table shown in Figure 11.

The supported logic symbols for the Inputs subsection for the Tests section are L, H, X, R, and F. These stand for logic LOW, logic HIGH, don't care (default too LOW), rising edge, and falling edge. For Outputs, the supported logic symbols are L, H, X, Z. Z stands for high-impedance (Hi-Z).

V. Testing Integrated Circuits

Test Procedure

To test an IC, follow the listed steps below:

1. Power the circuit using a USB-C 2.0 cable. The LED should light up.
2. Push up on the lever arm on the socket, so that it is vertical (Figure 16).

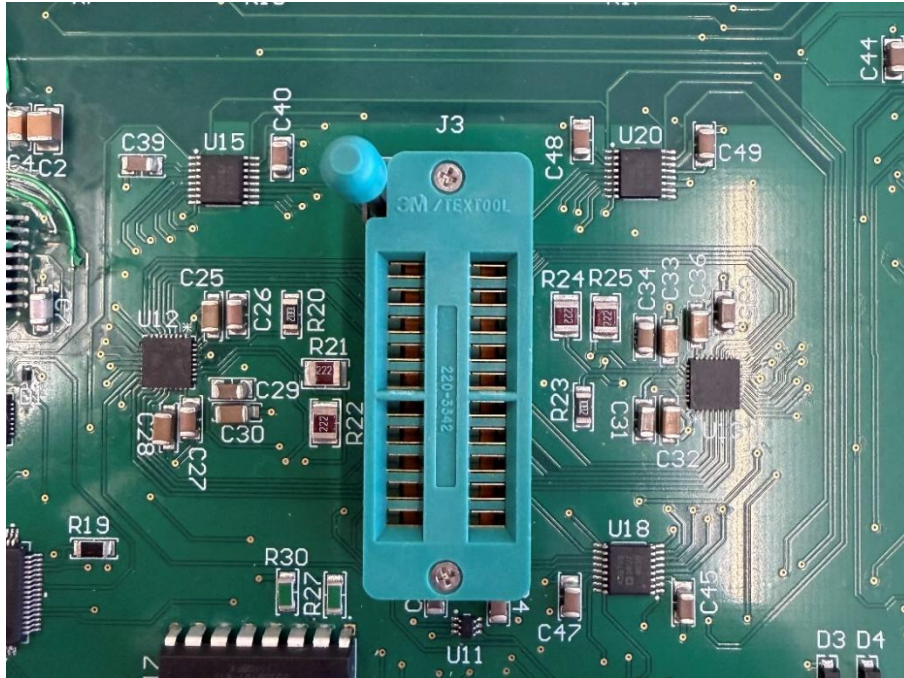


Figure 16. Demonstration of what the socket should look like after step 2.

3. Insert the IC and push down on the lever arm up so its horizontal. (Figure 17).

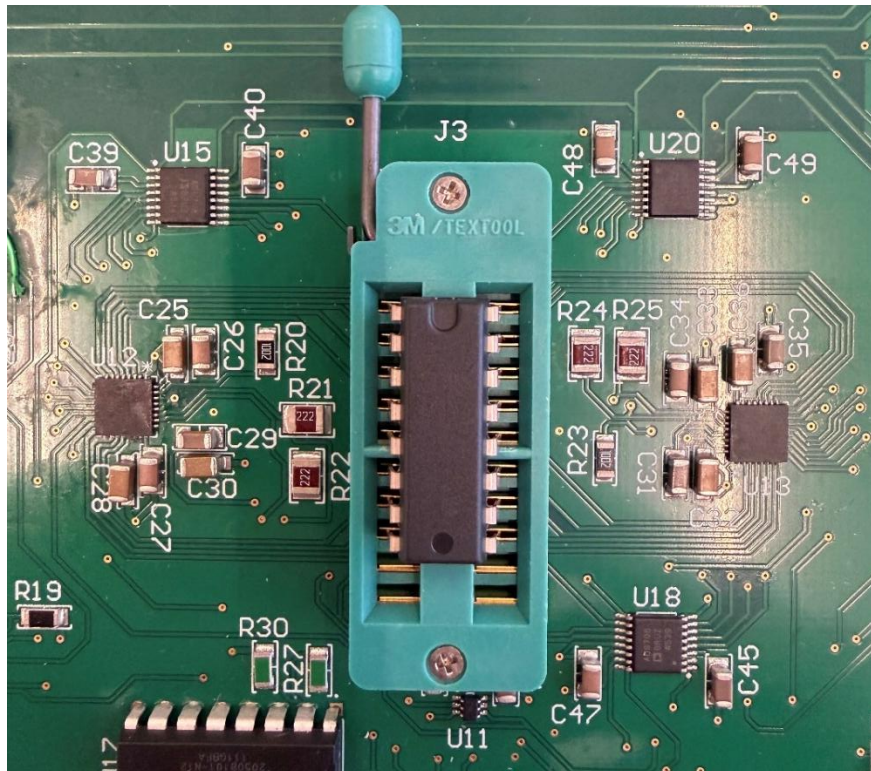


Figure 17. Demonstration of what the socket looks like after step 3.

4. In the application, open a test script for the IC
5. Run -> Run to execute the test script. DO NOT ATTEMPT TO REMOVE THE IC DURING TESTING.
6. Save the PDF report into the desired location.
7. Pull up on the lever arm and remove the IC.

Interpretation of Results

Pin Map

Pin Name	Pin	Pin Name	Pin
OE	1	1Q	2
1D	3	2D	4
2Q	5	3Q	6
3D	7	4D	8
4Q	9	CLK	11
5Q	12	5D	13
6D	14	6Q	15
7Q	16	7D	17
8D	18	8Q	19

Figure 18. Pin Map of the Digital IC under test.

Global Parameters

Parameter	Value
VCC Pin	20
GND Pin	10
VCC Voltage	5
Input Low	0.8
Input High	2
Output Low	0.33
Output High	3.84

Figure 19. Global Parameters of the Digital IC under test.

The test script configuration is defined through two key sections: Global Parameters and the Pin Map. Together, these provide a structured and hardware-independent way to control and evaluate the device under test (DUT).

The Global Parameters section establishes the core electrical settings required for testing, including the VCC and GND pin assignments, operating supply voltage, and logic threshold levels. These parameters define how the DUT is powered and how logical HIGH and LOW states are interpreted during testing. Since VCC and GND are automatically driven by the system, they do not need to be redefined elsewhere in the script, ensuring consistent and reliable operation.

The Pin Map section provides an abstraction layer that maps descriptive pin names to their corresponding physical socket positions. This allows test scripts to reference pins by function rather than numeric indices, improving readability and reducing the likelihood of errors. By separating logical naming from physical pin assignments, the same test logic can be reused across different ICs with minimal modification.

Together, these sections simplify test development by centralizing configuration, improving clarity, and enabling flexible adaptation to different devices and operating conditions.

Tests: IC PASS 16/16

Flip-Flop 1: PASS

Inputs			Outputs/Results	
OE	1D	CLK	1Q	
L	H	R	H	4.85 (H)
L	L	R	L	0.0 (L)
H	X	X	Z	1.59 (Z)

Figure 20. Example of passing test script for tri-state output IC.

Inputs			VCC	Outputs/Results	
OE	1D	CLK		1Q	
L	H	R	2.5	H	2.37 (H)
			3.3		3.15 (H)
			4.0		3.81 (H)
			4.5		4.43 (H)
			5		4.85 (H)

Figure 21. Example of passing test for IC tested at multiple voltage levels.

The outputs of all the tests are in the Tests section of the PDF report. In the header, there is an overall PASS or FAIL for the IC, and a fraction of the total tests that passed.

The results shown in Figures 20 and 21 demonstrate the correct functionality of the tri-state flip-flop IC under various input conditions and supply voltages. Overall, the device passed all 16 test cases, indicating that it operates within expected logic behavior.

Figure 20 validates the logical operation of the flip-flop under different input conditions. When the output enable (OE) is low (enabled) and the clock (CLK) experiences a rising edge, the device correctly latches the input data. Specifically, when D is high, the output Q transitions to a high state, measured at 4.85 V, which lies within the defined HIGH threshold. Conversely, when D is low, the output Q transitions to a low state, measured at 0.0 V, confirming proper LOW-level operation. When OE is high (disabled), the output enters a high-impedance (Hi-Z) state, indicated by a measured voltage of 1.59 V. This value falls between the defined HIGH and LOW thresholds, confirming that the output is not actively driven and is effectively floating, as expected for a tri-state output. These results demonstrate that the IC correctly performs data latching on the rising clock edge and properly transitions into Hi-Z mode when disabled.

Figure 21 demonstrates how the output voltage scales with varying supply voltages (VCC). As VCC increases from 2.5 V to 5 V, the measured output HIGH voltage increases proportionally, with values of 2.37 V at 2.5 V, 3.15 V at 3.3 V, 3.81 V at 4.0 V, 4.43 V at 4.5 V, and 4.85 V at 5 V. In all cases, the output remains within the defined HIGH logic threshold, confirming correct logic-level operation. These results indicate that the IC maintains proper functionality across a range of supply voltages, while also verifying that the system's voltage reference generation and measurement circuitry are operating accurately. Overall, the device under test (DUT) exhibits consistent and expected electrical behavior under varying operating conditions.

VI. Troubleshooting

Failed to Open Serial Port

Ensure device is detectable by host PC with the following information:

- **VID:** 10C4
- **PID:** EA60
- **Serial Number:**
554D4C2043617073746F6E652032352D333034204E5557432032303236

If not, ensure that the Silicon Labs CP210X driver is installed on your device.

https://www.silabs.com/interface/usb_bridges/classic/device.cp2102?tab=softwareandtools

The serial number can be programmed by downloading “AN721: CP210x/CP211x Device Customization Guide” through the same link.

Checksum

Ensure that firmware and Python executable have not been tampered with. Reflash and rebuild the issues persist.

If operating in a development environment, the firmware checksum can be updated in `software/serial_manager.py::Checksum` and update the variable `EXPECTED_CHECKSUM`. For the software checksum, ensure that `append_checksum.py` is called and the computed checksum is appended to the end of the .exe file. For SHA-256, the checksum is 32-bytes. During runtime,

the checksum is computed using all the bytes of the executable (except the last 32-bytes) and compared to the last 32-bytes of the executable.

Integrated Circuit Failed

Ensure that the IC is firmly placed in the socket and is level. Verify that the test script is written correctly. The number of the pins on the IC follows the standard counting for ICs, pin 1 is the top left, and increments counter-clockwise with pin 20 on the top right. This relevant for pins that are the right of the IC as they are likely shifted upwards by $20 - \text{number of pins on the IC}$.